

Article

Real-Time Edge Computing vs. GPU-Accelerated Pipelines for Low-Cost Microscopy Applications

Gloria Bueno ^{1,*} , Lucia Sanchez-Vargas ¹, Alberto Diaz-Maroto ¹, Jesus Ruiz-Santaquiteria ¹ , Maria Blanco ¹ , Jesus Salido ¹  and Gabriel Cristobal ² 

¹ VISILAB Group, Universidad de Castilla-La Mancha, 13001 Ciudad Real, Spain; lucia.sanchez@uclm.es (L.S.-V.); alberto.diazmaroto@uclm.es (A.D.-Z.); jesus.ralegre@uclm.es (J.R.-S.); maria.blanco@uclm.es (M.B.); jesus.salido@uclm.es (J.S.)

² Instituto de Óptica Daza de Váldes, Consejo Superior de Investigaciones Científicas (CSIC), 28006 Madrid, Spain; gabriel.cristobal@csic.es

* Correspondence: gloria.bueno@uclm.es

Abstract: Environmental microscopy is crucial for analyzing microorganisms, but traditional optical microscopes are often expensive, bulky, and impractical for field use. AI-driven image recognition, powered by deep learning models like YOLO, enhances microscopy analysis but typically requires high computational resources. To address these challenges, we present two cost-effective pipelines integrating AI with low-cost microscopes and edge computing. Both approaches use the OpenFlexure Microscope and Raspberry Pi devices. The first performs real-time inference with a Raspberry Pi 5 and Hailo-8L accelerator, while the second captures images with a Raspberry Pi 4, transferring them to a GPU-equipped desktop for processing. Using YOLOv8, we evaluate their ability to detect phytoplankton species, including cyanobacteria and diatoms. Results show that edge computing enables accurate, efficient, and low-power microscopy analysis, demonstrating its potential for real-time environmental monitoring in resource-limited settings.

Keywords: AI-driven real-time microscopic image processing; edge computing; low-cost microscopy; open flexure; phytoplankton identification



Academic Editors: Wen-Jing Zhou, Hongbo Zhang and Yuyin Zhou

Received: 29 December 2024

Revised: 17 February 2025

Accepted: 20 February 2025

Published: 26 February 2025

Citation: Bueno, G.; Sanchez-Vargas, L.; Diaz-Maroto, A.; Ruiz-Santaquiteria, J.; Blanco, M.; Salido, J.; Cristobal, G. Real-Time Edge Computing vs. GPU-Accelerated Pipelines for Low-Cost Microscopy Applications. *Electronics* **2025**, *14*, 930. <https://doi.org/10.3390/electronics14050930>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Optical microscopy is a widely used technique in biology, medicine, and materials science for observing samples at the microscale. It is essential for identifying microorganisms in environmental monitoring (e.g., cyanobacteria or diatoms), diagnosing diseases (e.g., parasites or cancer cells), and analyzing materials (e.g., metals or polymers) [1–6], among other applications. However, traditional optical microscopes are expensive, bulky, and require a stable environment to operate correctly. They are usually unsuitable for on-site work, where samples need to be analyzed in the field or in remote locations such as water bodies or rural areas. Additionally, the high cost of traditional microscopes makes them inaccessible to low-income communities or developing countries, where the need for microscopy is high but resources are limited [7].

To address these challenges, low-cost and portable microscopy solutions have been proposed and developed in recent years. For instance, the Foldscope is an ultra-low-cost, origami-based microscope capable of imaging in brightfield, darkfield, and fluorescence modes, specifically useful for educational purpose. It has been distributed to over 150 countries, increasing science accessibility worldwide [8,9].

Another example is the development of a low-cost, portable microscope with built-in vision AI to automate microscopic diagnosis of diseases in remote rural settings. This device, called MAIScope, is designed to tackle the practical constraints of malaria diagnosis in rural areas [10,11].

CellScope [12] is another example of a low-cost microscope that uses a smartphone as a camera to capture images of samples. In this case, the use of more complex optical components allows for improved image quality and resolution.

Versatility is another key aspect when considering this type of microscopy solution. The ability to adapt the microscope to different types of samples and applications is essential for its widespread use. In this sense, the UC2 project [13] proposes a low-cost open standard platform to build custom microscopes, allowing for standard brightfield microscopy and also fluorescence microscopy by adding a few components. This platform is based on a modular design philosophy; it is composed of 3D-printed parts and of-the-shelf components.

Other approaches like MicroHikari3D [14] are focused on applications that need whole-slide scanning. In this case, a motion stage with larger travel distances is required to scan the whole slide. The proposed system is adapted from a commercial 3D printer, which allows for a low-cost solution with high precision and repeatability. The system is controlled by a Raspberry Pi (RPi) 4 and includes interesting features like autofocus, different illumination modes, and also integration with mobile devices for remote control and deep learning-based image analysis.

One of the most popular open-source microscopy solutions is the OpenFlexure microscope [15], to be manufactured using 3D printing. The OpenFlexure microscope is the one used in this study. Its structure incorporates a precise flexure mechanism that allows for accurate positioning and focusing of samples. This modular and customizable design supports various optical configurations, ranging from inexpensive webcam lenses to high-quality laboratory microscope objectives, making it adaptable to multiple applications in biology, medicine, and materials science [16]. A notable feature of the OpenFlexure Microscope is its automation capability. By integrating motors and controllers, it is possible to automate focusing and sample movement, enabling the acquisition of high-resolution images and the execution of complex analyses. Moreover, its open-source design allows the scientific and educational community to collaborate on its development and adaptation, promoting access to advanced microscopy tools in resource-limited settings. This project has been implemented in various regions, including Tanzania and Kenya, where over 100 microscopes have been produced for educational, scientific and clinical applications [17].

These innovations aim to democratize access to microscopy, enabling its use in environmental monitoring, medical diagnostics, and materials analysis in contexts where traditional microscopes are not viable. In general, these solutions are based on open hardware and software, following the do-it-yourself (DIY) philosophy to reduce costs and increase accessibility [18]. They demonstrate that local manufacturing can be a viable alternative to international supply chains, which, as previously mentioned, are often costly and lack flexibility for field applications in microscopy.

On the other hand, the use of deep learning-based image processing techniques has gained popularity in recent years for computer vision applications, such as object detection, image classification or image segmentation. These techniques have shown excellent performance in a wide range of applications, including microscopy. In particular, deep learning models have been used for the identification and classification of microorganisms in microscopy images, such as parasites in smear samples [3] or diatoms in water samples [19,20]. Unfortunately, the deployment of this type of deep learning models usu-

ally requires powerful hardware resources for inference, such as Graphical Processing Units (GPUs) or Tensor Processing Units (TPUs) with high memory capacity and processing power. This makes the deployment of deep learning models in low-cost and portable microscopy solutions challenging as these devices usually have limited computational resources. Recently, different types of hardware accelerators have been proposed to address this challenge, such as the Google Coral TPU [21] or the HAILO-8L accelerator [22].

The main goal of the OpenFlexure project is to develop an open-source, lab-grade microscope that costs significantly less than traditional microscopes, aiming to make microscopy accessible to everyone. Optical microscopy is essential for tasks such as disease detection and water quality monitoring, which are particularly critical in developing regions. However, deep learning models for object recognition in microscopy images require high computational power, which is challenging to achieve with edge devices. The integration of the HAILO-8L GPU accelerator into edge devices enables real-time inference of deep learning models, addressing this limitation.

In this context, this paper presents two main contributions aimed at addressing the challenges of deploying deep learning models in low-cost microscopy solutions, particularly in response to the need for microscopes with edge computing capabilities for in situ applications. Such solutions are critical for field-based tasks like environmental analysis, including water quality characterization, which represents a key use case in this study. To evaluate their effectiveness, these two methodologies have been developed and compared:

- (a) The development of a software pipeline for image processing of microscopy images using deep learning architectures implemented on a Raspberry Pi 5, equipped with the HAILO-8L accelerator, showcasing the potential of edge computing for portable, low-cost systems.
- (b) A complementary approach involving a second software pipeline for image processing of microscopy images, using deep learning architectures on a standard desktop computer with GPU acceleration, providing a benchmark for assessing the performance and scalability of the edge computing approach.

The rest of this paper is organized as follows. Section 2 summarizes the related work in the field. Section 3 describes the proposed methodologies. Section 4 explains the experiments conducted to evaluate the proposed method. Finally, Section 5 concludes the paper and discusses potential lines of future work.

2. Related Work

In contrast to traditional optical microscopes, low-cost and portable solutions have been proposed in recent years to address challenges related to accessibility and cost. These systems generally meet the following requirements: lightweight design, simple optical components, ease of assembly and use, low energy consumption, and independent imaging systems [7]. Versatility is another key aspect when considering this type of microscopy solutions. The ability to adapt the microscope to different types of samples and applications is essential for its widespread use.

Various low-cost microscopy systems have already been discussed in the previous section. In this work, we have chosen to use OpenFlexure, as it is one of the most popular and flexible options available [15,23].

There are different versions of the OpenFlexure microscope, allowing for different configurations and applications, such as brightfield, darkfield, rheinberg, differential interference contrast (DIC), or ptychography. In addition, it can be configured as an inverted or upright microscope, depending on the application requirements. The hardware is controlled by a RPi 4/5 and a motor driver controller for the stepper motor management.

In general, the low-cost microscopy solutions mentioned earlier have limited computational resources as they are typically based on single-board computers like the Raspberry Pi, which are designed to perform basic tasks such as stepper motor control, image acquisition, and visualization and simple image processing. However, advanced microscopy applications often require more sophisticated computer vision techniques, such as object detection, image classification or image segmentation. These techniques generally rely on deep learning-based methodologies, which have demonstrated excellent performance across a wide range of applications. Nevertheless, such architectures typically require powerful hardware resources for inference, such as GPUs or TPUs, making their deployment on these low-cost devices particularly challenging.

To address this challenge, various hardware accelerators have been proposed in recent years. One example is the Google Coral TPU [21], a compact TPU-based accelerator that can be connected to other devices, including the RPi, via a USB port. This accelerator enables the deployment of deep learning models compiled using the TensorFlow Lite framework. The coprocessor is capable of performing 4 trillion int8 operations per second (TOPS) with a power efficiency of 2 TOPS/W, consuming only 2 watts. However, its software support is limited and has been discontinued, making it challenging to integrate into new projects.

Another example of a hardware accelerator is the HAILO-8L [22], which delivers 13 TOPS with a typical power consumption of 1.5 watts, achieving an impressive efficiency of 8.7 TOPS/W. This accelerator enables real-time, low-latency, and highly efficient deep learning inference. When used with a RPi 5, the HAILO-8L can be connected via a HAT (Hardware Attached on Top) board using the M.2 PCIe interface. Its software support allows the utilization of out-of-the-box deep learning architectures, such as MobileNet [24], ResNet [25], and the YOLO family [26]. Additionally, it supports the compilation of custom models created with popular deep learning frameworks like TensorFlow, PyTorch, Keras, or ONNX.

3. Methodology

In this work, we propose and compare two different software pipelines for image processing in low-cost microscopy solutions using deep learning architectures. Both pipelines are applied to the OpenFlexure microscope, which, as previously mentioned, was chosen for its versatility, low cost, and open-source nature [23].

Phytoplankton recognition in water samples is crucial for monitoring water quality and preventing harmful cyanobacterial blooms caused by water eutrophication. There are thousands of diatom and cyanobacteria classes, yet experts can only recognize a limited number of them. Additionally, cyanobacteria taxonomy is continuously evolving, making classification increasingly challenging for experts. Manual labeling is a time-consuming process, and new, previously unknown species may appear, further complicating classification. Phytoplankton samples are highly diverse; in some cases, images contain crowded fields with multiple specimens and debris, requiring experts to manually count the number of specimens per species for analysis.

The YOLO architecture has demonstrated outstanding performance in phytoplankton recognition, achieving high accuracy and speed, making it a strong candidate for real-time phytoplankton detection applications [27,28]. YOLOv8 introduces enhancements in small object detection and supports image classification tasks, making it particularly suitable for this study. Another key factor in model selection was compatibility with the HAILO device. Although HAILO supports multiple frameworks (including PyTorch, TensorFlow, ONNX, and Keras), models must be compiled into HAILO's binary format, an optimization process that converts standard frameworks into a HAILO-executable format for deployment.

3.1. Prototype Description

The OpenFlexure project offers two main variants based on the motion configuration: the OpenFlexure Microscope and the OpenFlexure Delta Stage. The main difference lies in how motion is managed. In the original OpenFlexure Microscope, independent actuators control movement along each axis, with the sample stage moving in the X and Y directions and the optics module moving along the Z-axis. In contrast, the OpenFlexure Delta Stage fixes the optics module in place while the sample stage moves in all three axes (X, Y, and Z). This design makes the Delta Stage particularly suitable for more advanced optical setups, such as Differential Interference Contrast (DIC) or fluorescence microscopy, as the stationary optics allow for more precise and stable configurations. For this work, both versions were considered, namely, the OpenFlexure Microscope version 7.0.0 and the OpenFlexure Delta Stage version 1.2.1. In Figure 1, the assembled OpenFlexure microscope and OpenFlexure Delta Stage microscope, 3D-printed in the VISILAB laboratory for this study, are shown. Table 1 shows the main specifications of both versions, including the hardware components and the software used.

The OpenFlexure project provides comprehensive resources for replication and customization. The full bill of materials (BOM) and step-by-step assembly instructions for both versions can be found on the OpenFlexure project website (<https://openflexure.org/>, accessed on 6 January 2025) including the necessary Standard Tessellation Language or Stereolithography (STL) files for the 3D-printed parts. The OpenFlexure project also provides a custom software application for the control of the microscope, called OpenFlexure Connect, which allows for the control of the stepper motors, the camera and the illumination system, as well as the visualization of the images captured by the camera. The software is designed to offer an intuitive interface, making it easy to adjust settings such as focus and lighting for optimal image quality. Additionally, OpenFlexure Connect supports various customization options, allowing users to fine-tune the system according to their specific needs.

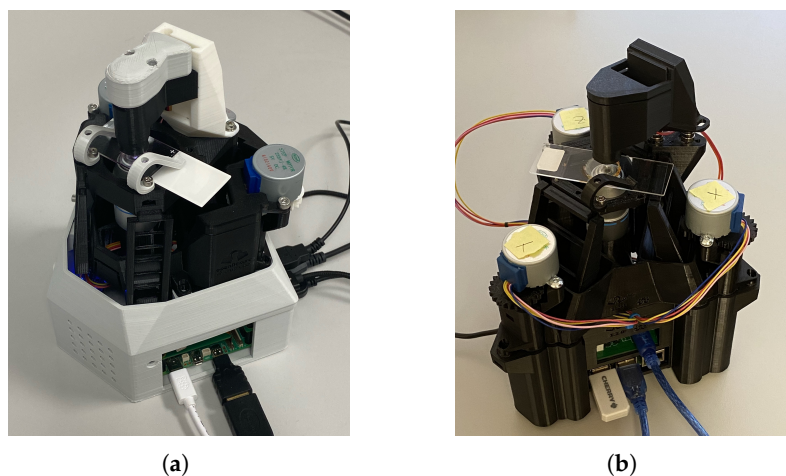


Figure 1. Assembled OpenFlexure prototypes: (a) OpenFlexure Microscope; (b) OpenFlexure Delta Stage.

Table 1. Specifications of the OpenFlexure microscope.

Component	Version 7.0.0	Version Delta 1.2.1
Base hardware	Raspberry Pi 5 + Hailo-8L M.2	Raspberry Pi 4
Operating system	Raspbian 12	Raspbian 10
Camera V2.1	Raspberry Pi (IMX219)	Raspberry Pi (IMX219PQ)
Camera software	libcamera 0.3.2	Picamera 1.13
Objective lens	40/0.65 (40X)	40/0.65 (40X)
Motor driver controller	Sangaboard v0.5.4	Sangaboard v0.2
Steeper motor	28NYJ-48 5V DC	28BYJ-48 5V DC
Base software	Custom application	OpenFlexure Connect 4.0.1

3.2. Pipeline 1: Edge Device with HAILO-8L Accelerator

The first software pipeline is based on the first prototype, the OpenFlexure Microscope, which includes a RPi 5 and the HAILO-8L accelerator. In this pipeline, all tasks are executed directly on the edge device, including stage movement, image acquisition, image processing, visualization, and, with the support of the HAILO-8L accelerator, the inference of deep learning models for the detection and classification of microorganisms in the images.

To this end, a custom software application was developed using the Python programming language along with libraries such as OpenCV, PyQt, Picamera, and HailoRT. The main interface of the application is shown in Figure 2. Below is a summary of the key features the application offers to the user:

1. **Model and Microscope Flexibility:** Although this software application was developed specifically for this experiment, it is compatible with any detection model that can be compiled into Hailo’s Model Representation and supports various microscope setups. Users can select the model file for inference (along with a JSON file specifying class names) and adjust thresholds for “Confidence” or “Score” and Intersection Over Union (IoU) to filter overlapping detections. These changes require a quick and nearly imperceptible reset to take effect.
2. **Frame Rate Control:** The frame rate can be limited to reduce CPU load, offering better control over system performance.
3. **Video Recording:** The application includes options to record video in the desired format, with the ability to pause and resume recording at any time.
4. **Stage Control:** The OpenFlexure motors can be controlled to move the stage along the XY axes and the optics module along the Z axis, providing precise adjustments.
5. **Real-Time System Feedback:** The application displays real-time feedback on the pipeline and system status, including CPU load, frames per second, and the stage’s current coordinates.
6. **Input Device Selection:** Users can choose the desired input device, enabling compatibility with different setups and other types of cameras.

Support for automatic object identification in images was implemented using the YOLOv8 architecture [29], which was compiled with the Hailo Dataflow Compiler Software Package (3.30.0 version). YOLOv8 was chosen for its high performance in terms of both speed and accuracy, making it especially suitable for real-time applications and resource-constrained environments [30]. During the compilation process, the trained YOLOv8 model is converted into a format compatible with the HAILO-8L accelerator. Additionally, an optimization process is applied to adapt the model to the Hailo device, enhancing performance while maintaining the original architecture and functionality as closely as possible.

The image processing pipeline developed for this implementation involves several steps. First, the Camera Module is initialized in preview mode using the RPi’s Picamera library, which configures the hardware for image acquisition. The Camera Module captures

images at a consistent frame rate (typically 30 FPS, though this may vary depending on the video configuration) and transfers the image buffer via the four-lane MIPI interface of the RPi 5. This direct hardware interface ensures minimal latency and efficient data transfer. The captured image array is then processed by the RPi's hardware ISP (Image Signal Processor), which performs tasks such as noise reduction and color correction, eliminating the need for additional software intervention during these preprocessing stages.

Next, a flat-field correction algorithm is applied to the image to correct any uneven illumination or sensor-specific artifacts. The corrected image is then resized to match the input dimensions required by the deep learning model. A copy of the original image is retained in memory for later use in the user interface, where it will be displayed alongside the inference results. This step ensures that the visual quality of the displayed image is not compromised by resizing or preprocessing.

The resized image is sent via a shared buffer to a separate process dedicated solely to running inferences on incoming images. Inference is performed using Hailo's Runtime and library, leveraging concurrency to fully utilize the RPi 5's multicore architecture. This approach ensures a stable frame rate even when using larger models.

Once inference is completed for an image, the results are sent back through the buffer to the primary process, where bounding boxes are drawn on the original image. While processing inference and image display in separate concurrent processes significantly enhances performance, it may introduce a slight delay between the displayed image and the overlaid bounding boxes. However, since this is a real-time video preview application, the difference of two or three consecutive frames is generally negligible. As a result, this approach effectively balances performance with the hardware limitations of the RPi 5.

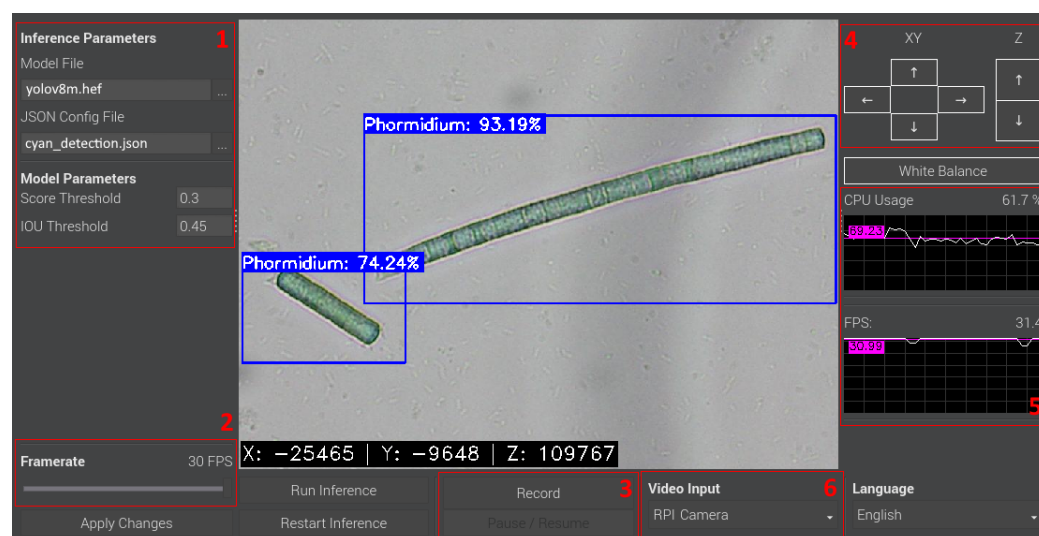


Figure 2. Main interface of the software application developed for the first pipeline: Edge device with HAILO-8L accelerator. Where: 1 shows the model and microscope flexibility; 2 shows the frame rate control; 3 shows the video recording; 4 refers to the stage control; 5 shows the real-time system feedback and 6 shows the input device selection.

3.3. Pipeline 2: Desktop Computer with GPU Acceleration

Although the pipeline 1 enables direct control of the microscope and real-time deep learning inference on the edge device, leveraging the RPi5 and the HAILO-8L accelerator, its computational resources can be inherently limited for certain demanding applications. Additionally, the HAILO-8L accelerator supports only a restricted set of deep learning architectures, making it challenging to deploy custom models or more complex architectures. Nevertheless, as demonstrated in this study, the results obtained from the pipeline 1 with

the HAILO-8L accelerator on the RPi5 can be competitive, even in scenarios traditionally dominated by higher-performance desktop setups.

To further evaluate the performance of the edge device and highlight its competitiveness against more traditional methods, the second pipeline was introduced as a benchmark. This pipeline performs image processing on a standard desktop computer equipped with GPU acceleration, providing a direct comparison of the capabilities and trade-offs between the edge-based and desktop-based solutions.

In the pipeline 2, the OpenFlexure microscope is used as a remote sensor to capture images, which are then transmitted to the desktop for processing. The software application for this pipeline was developed using the OpenFlexure client and the OpenFlexure Python library, in combination with other libraries such as PyQt, OpenCV, MMCV-full, and Pycocotools.

The OpenFlexure client library provides extensive functionality, enabling users to control various microscope features, including stage movement, image capture, autofocus, white balance, and flat-field correction. While the PiCamera typically supports a frame rate of 30 FPS, the actual frame rate in this setup is determined by network speed, introducing a slight delay between the microscope's actions and the camera stream. This delay is minimal but may impact applications requiring immediate feedback. The stream resolution is set to 832×624 pixels, ensuring sufficient image quality for the tasks at hand.

The application includes several modules, as shown in Figure 3. These modules are as follows:

1. **Detection module:** This allows users to select the detection model and set the detection threshold.
2. **Classification module:** This enables classification at different levels, depending on the structure of the object being analyzed. In our study's use case of water quality assessment, the module allows genus- and species-level classification for diatoms and order and genus-level classification for cyanobacteria, providing flexibility for diverse biological applications.
3. **Inference module:** This provides controls to start and stop inference processes and select the device used for inference.
4. **Editing modules:** These allow for the customization of annotation visualization and saving formats.
5. **Annotation module:** This allows us to choose the path for saving the annotations file and to choose the format from Pascal VOC, YOLO, COCO, or basic JSON.
6. **OpenFlexure module:** This enables remote control of the microscope, detects network-connected devices, and allows for their selection for camera streaming. Once transmission is activated, it provides various options, including image capture, white balance adjustment, autofocus, and motor control.

The menu bar provides a range of file-handling options, including the ability to open individual images or entire folders, as well as to change the annotation folder where results are saved. It also offers view options for browsing and previewing uploaded image files, allowing users to navigate through datasets conveniently. Export options are available to extract and analyze inference results, enabling efficient post-processing and integration with other tools.

In addition to these functionalities, the menu bar includes the OpenFlexure module, which facilitates establishing a connection with the microscope. This module provides access to various features of the microscope, such as stage movement for precise positioning, image capture to acquire snapshots of the sample, and autofocus to ensure clarity and sharpness in the captured images. These features make the application versatile and user-friendly, supporting both real-time operation and detailed analysis.

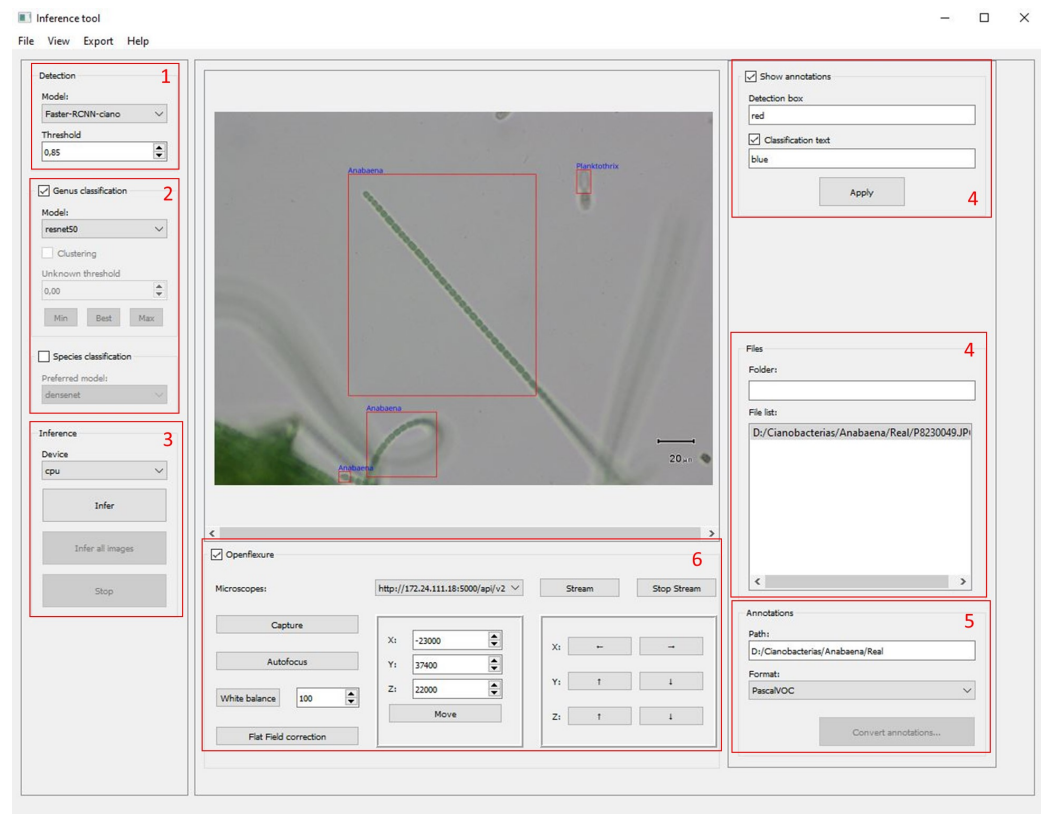


Figure 3. Main interface of the software application developed for the second pipeline: desktop computer with GPU acceleration. Where: 1 shows the detection module; 2 is the classification module; 3 is the inference module; 4 shows the editing module; 5 refers to the annotation module; and 6 is the OpenFlexure module.

The desktop GPU used in this pipeline consists of 2 NVIDIA®RTX™6000 ADA modules with a memory of 48 GB. This GPU delivers the features, capabilities, and performance to meet Ai-driven workflows with a single precision performance of 91.1 TFLOPS. It is capable of perform tasks that require extensive memory resources such as training of Deep Learning models.

OpenFlexure Inference Pipeline: When the OpenFlexure module is activated, the software searches the network for microscopes using multicast DNS services, which return the IP addresses of detected microscopes. Once a valid IP address is selected, the microscope's controls are unlocked. By pressing the stream button, users can access functions like stage movement, image capture, and autofocus, enabling them to navigate to the desired sample view and adjust the brightness.

The Camera Module begins capturing images continuously until the capture button is pressed. This action captures a single image and activates all inference options. The pipeline then processes the image as follows:

- **Image Preprocessing:** The image array is resized to match the input dimensions required by the detection model.
- **Detection:** The detection model generates bounding boxes, which are saved to a temporary file.
- **Classification:** The bounding boxes are resized to match the input dimensions of the genus/species classification networks, and the classification process is executed.

Once inference is complete, the results, including bounding boxes and classification labels, are displayed over the original image. Additionally, an annotation file is created and

automatically saved to the specified path. The entire process is performed by the selected device, either the CPU or GPU (CUDA), if available.

3.4. Dataset

Several training processes were conducted to evaluate model performance for this application using different dataset preparation techniques. After analyzing the distribution of available images, we selected the most abundant and diverse classes for training. Results from previous experiments indicated that YOLOv8 struggled to accurately detect certain classes, such as *Nostoc* and *Planktothrix*. This difficulty was likely due to the limited diversity of images available for these classes. Consequently, we removed them from the dataset to minimize class confusion and focus on classes with better detection performance.

We also experimented with various data augmentation techniques to increase the size of the cyanobacteria dataset but found that the generated images were of very poor quality. Due to the small dataset size, traditional methods, such as rotations, horizontal, and vertical flips and scaling produced highly similar images, leading to overfitting.

To address this, we explored machine learning-based approaches, specifically the StyleGAN2-ADA architecture [31], a generative adversarial network (GAN) that generates images based on the probability distribution of the cyanobacteria dataset. The model was trained independently for each cyanobacteria class, generating approximately 400 images per class. We then assessed the perceptual quality metrics of these images and removed the lowest-quality ones. Ultimately, around 150 images were added to each class, but this did not result in a significant improvement in model performance. Consequently, we decided to discard this approach for this experiment.

The results showed that a larger dataset led to a significant performance improvement only in larger architectures capable of capturing more specific features. The cyanobacteria dataset (716 images) is relatively small compared to the diatom dataset (4953 images), which explains the minimal performance improvement in cyanobacteria models across different architectures compared to diatom models.

4. Experiments and Results

These pipelines have been demonstrated for the microscopic identification of diatoms and cyanobacteria, including their detection and classification, with a focus on water quality assessment. The identification of microscopic organisms such as diatoms and cyanobacteria plays a crucial role in environmental monitoring and public health. It helps assess water quality by detecting cyanobacterial blooms, early signs of eutrophication and monitoring ecosystem health. This identification is also vital for public health, as it enables the detection of toxin-producing species, allowing for early warnings for water safety purposes and the targeted treatment of contaminated water supplies.

In ecological research, it provides insights into community dynamics, biodiversity, and the role of diatoms and cyanobacteria in aquatic ecosystems. Additionally, it supports the sustainable management of water resources, including reservoirs, lakes, and aquaculture systems, by controlling harmful blooms and mitigating their impact on productivity. Diatoms are also used in various fields, such as agriculture, pharmacy, and biochemistry, where they are employed in biofuel production. Ultimately, this identification promotes sustainable water management and helps protect both ecosystem and human health.

4.1. Materials

The datasets used consist of microscopy images of cyanobacteria and diatom samples. Cyanobacteria images were taken by the biology department of the Autonomous University of Madrid (UAM), which consists of environmental and cultivation samples.

The environmental samples were collected with a special net for phytoplankton with a pore size of 20 mm in reservoirs in several Spanish rivers such as Tajo, Miño, and Júcar. The samples were taken during the months of May to October, when the flowering period occurs. The complete dataset has 36 genera, of which 6 were selected for this task due to the imbalance present between the classes. The images include three orders of cyanobacteria, with two genera selected from each order, resulting in a total of six genera. Specifically, the dataset contains *Anabaena* and *Dolichospermum* from the order *Nostocales*; *Chroococcus* and *Woronichinia* from the order *Chroococcales*; and *Phormidium* and *Lyngbya* from the order *Oscillatoriales*. Figure 4 shows example images for each class in the dataset.

The dataset is organized using a structure called *Stacker*, which groups images into stacks, with each stack corresponding to a specific class. Images within the same stack exhibit a structural similarity of 70% or higher.

As can be seen in Figure 5, the images belonging to the same stack represent a sequence in which different points of the sample are in focus. Stacks of living samples can also show the movement of cyanobacteria, which may reduce the structural similarity between images.

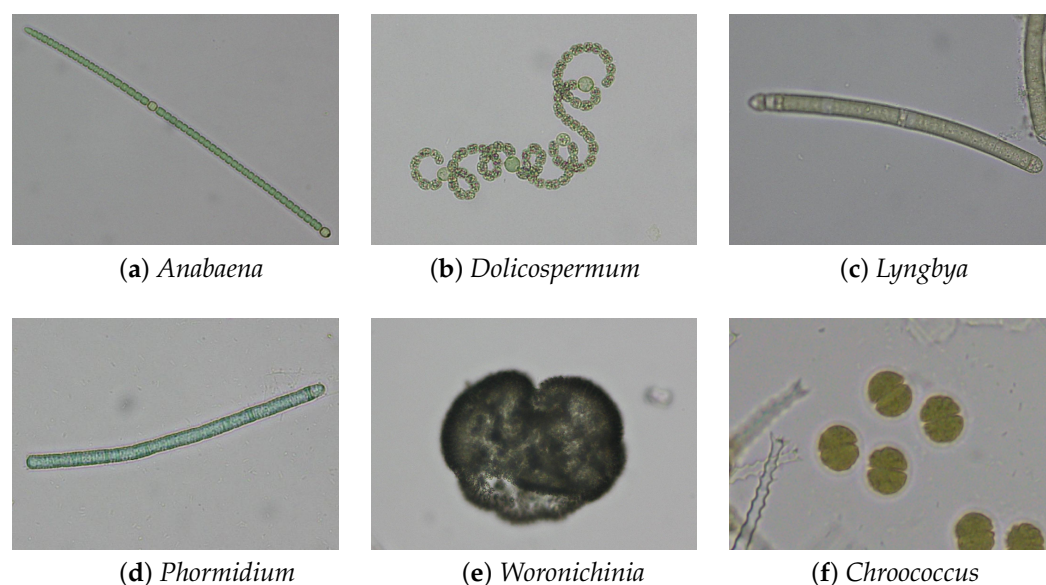


Figure 4. Real images at 40× of the cyanobacteria genera used. Images (a–d) are considered filamentous type of cyanobacteria; while images (e,f) are considered colonies of cells. Genera of the same order share same morphology.



Figure 5. Two images that belong to the same stack from the cyanobacteria genera *Anabaena*. The images were obtained at 40× magnification.

To achieve the identification objective, image labeling was performed manually using the MATLAB (R2023b version) Image Labeler application, which allows for the annotation of regions of interest (ROI) in each image. These annotations are initially generated in COCO format, the default output of Image Labeler. To use them with the models discussed in Pipeline 1 and Pipeline 2, a conversion to the YOLO annotation format was performed.

Freshwater diatom samples were collected from the Duero River by Aqualitas. Images were captured using a Westbury Brunel SP40 microscope with bright-field microscopy. A total of 100 species were classified and grouped into 36 genera, of which 9 genera were selected for this study: *Aulacoseira*, *Cocconeis*, *Cymbella*, *Diatoma*, *Epithemia*, *Gomphonema*, *Melosira*, *Planothidium*, *Staurosira*, and *Tabellaria*. The images contain annotated diatoms from the same class and unannotated diatoms from different classes. Figure 6 shows examples from several river basins in Spain.

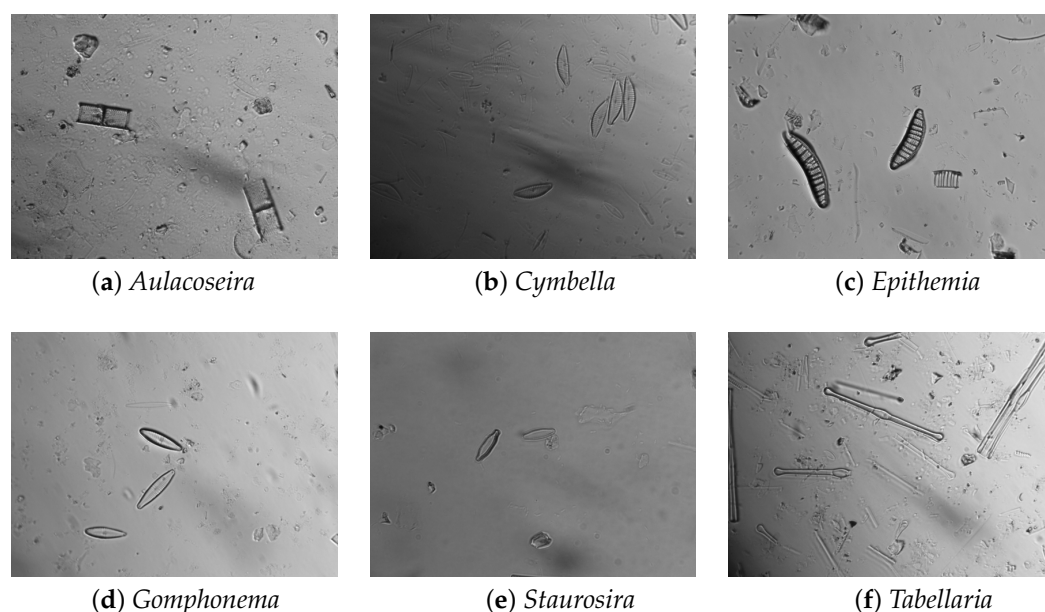


Figure 6. Real images at 40× of the diatom genera used: (a) two full specimens; (b) five full specimens and another diatom in the background; (c) two specimens; (d) three species from the *Gomphonema* genus, but different species; (e) two full specimens; (f) two complete specimens in different positions, with parts showing an advanced life cycle stage.

The images in the diatom dataset were already annotated in XML format, so a conversion to the YOLO annotation format was performed. Several stacks existed within some genera, and the images within the same stack exhibit an average structural similarity of 90% or higher.

As can be seen in the Figure 7, the images belonging to a same stack represent the same image where different parts are focused. There are no live samples in the diatom dataset so the structural similarity is higher.

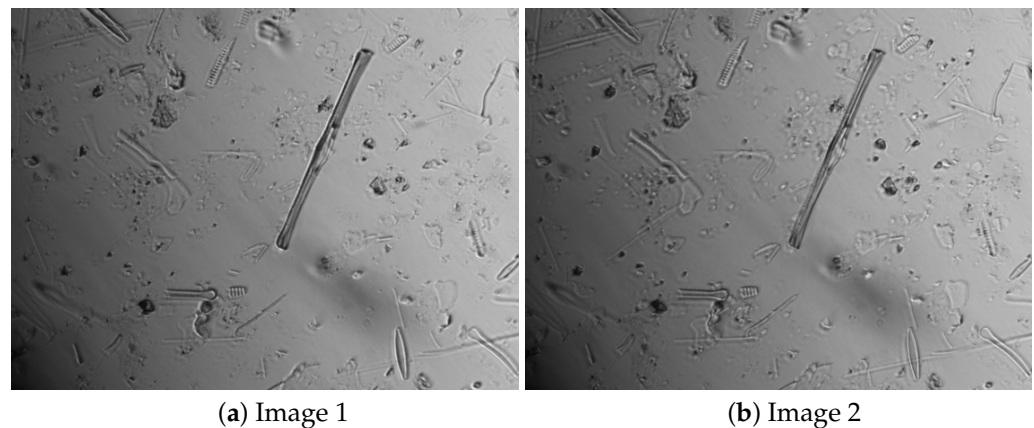


Figure 7. Two images that belong to the same stack from the diatom genre *Tabellaria*. The images were obtained at 40× magnification.

4.2. Experimental Setting

After completing the dataset creation process for both use cases cyanobacteria and diatom, they were subdivided following the 70–20–10 rule, with some variation depending on the number of images in each stack. Specifically, 70% of the instances were allocated for training, 20% for testing, and 10% for validation. The total number of instances per class and the corresponding number of images after this subdivision are shown in Table 2 for cyanobacteria and Table 3 for diatom.

Table 2. Cyanobacteria dataset split for training, validation, and testing.

Order	Genera	Train	Validation	Test	All
Nostocales	<i>Anabaena</i>	79	12	22	113
	<i>Dolicospermum</i>	74	11	20	105
Chroococcales	<i>Chroococcus</i>	114	16	32	162
	<i>Woronichinia</i>	42	6	11	59
Oscillatoriales	<i>Phormidium</i>	67	10	20	97
	<i>Lyngbya</i>	126	18	36	180
Total		502	73	141	716

Table 3. Diatom dataset split for training, validation, and testing.

Genera	Train	Validation	Test	All
<i>Aulacoseira</i>	143	20	40	203
<i>Cocconeis</i>	599	86	171	856
<i>Cymbella</i>	636	91	181	908
<i>Diatoma</i>	192	27	54	273
<i>Epithemia</i>	668	95	190	953
<i>Gomphonema</i>	473	68	135	676
<i>Melosira</i>	309	44	88	441
<i>Planothidium</i>	142	20	40	202
<i>Staurosira</i>	140	20	40	200
<i>Tabellaria</i>	169	24	8	241
Total	3471	495	987	4953

4.3. Results

Several experiments were conducted to evaluate the performance of the proposed pipelines for the automatic identification of phytoplankton, including both cyanobacteria

and diatoms, in microscopy images. The experiments utilized the datasets previously described and the YOLOv8 architecture [29] for object detection. The YOLOv8 architecture offers various versions, each differing in the number of layers and parameters. In this study, four versions were considered—YOLOv8-s, YOLOv8-m, YOLOv8-l, and YOLOv8-x—to assess their performance in terms of both accuracy and speed.

Mean Average Precision (mAP) was chosen as the evaluation metric for the experiments, as it is a standard measure in object detection tasks. mAP is calculated as the mean of the Average Precision (AP) for each class, where AP represents the area under the precision–recall curve. For this study, mAP was computed at different Intersection over Union (IoU) thresholds: mAP50, which evaluates precision at an IoU threshold of 0.5; and mAP50:95, which represents the mean average precision across IoU thresholds ranging from 0.5 to 0.95 in increments of 0.05.

Regarding the training process, all architectures were trained with consistent hyperparameters and the same strategy to ensure a fair comparison. The Ultralytics YOLOv8 repository [29] was used for training. Training was conducted for 20 epochs with a batch size of 32 and the learning rate was set to 0.01 using the SGD optimizer. The process was carried out on a desktop computer equipped with an NVIDIA RTX 6000 GPU. All input images were resized to 640×640 pixels to meet the model’s requirements.

4.3.1. Cyanobacteria Results

First, the evaluation metrics for the different versions of the YOLOv8 architecture were calculated using the test split. Tables 4 and 5 present the *mAP50* and *mAP50:95* scores for each of the six cyanobacteria genera, as well as the mean values across all classes, calculated with the two pipelines. The results show that the YOLOv8-l architecture achieved the best average performance in terms of accuracy for both methodologies.

For Pipeline 1, the YOLOv8-l architecture obtained a *mAP50* of 0.791 and a *mAP50:95* of 0.540, yielding an average of 0.665. Similarly, for Pipeline 2, it achieved a *mAP50* of 0.800 and a *mAP50:95* of 0.544, with an average of 0.672. However, the differences between the YOLOv8 variants are minimal. For instance, the YOLOv8-s architecture achieved a *mAP50* of 0.792 and 0.794 for Pipeline 1 and Pipeline 2, respectively, along with a *mAP50:95* of 0.532 for both pipelines, resulting in averages of 0.662 and 0.663, respectively.

These results can be attributed to the small size of the dataset and the high similarity between the cyanobacteria genera, particularly those within the same order. Such similarity may make it more challenging for the models to distinguish between genera, thus affecting their overall performance. This underscores the importance of expanding both the size and diversity of the dataset to improve model accuracy and robustness. Increasing the variety of genera, as well as incorporating additional environmental conditions or sample types, could further enhance the models’ ability to generalize and improve their performance in real-world scenarios.

Table 4. Object identification results for different cyanobacteria genera using the YOLOv8 architecture on the HAILO-8L accelerator (Pipeline 1).

Order	Genera	YOLOv8-s		YOLOv8-m		YOLOv8-l		YOLOv8-x	
		mAP50	mAP50:95	mAP50	mAP50:95	mAP50	mAP50:95	mAP50	mAP50:95
Nostocales	<i>Anabaena</i>	0.790	0.501	0.815	0.546	0.863	0.539	0.818	0.546
	<i>Dolicospermum</i>	0.844	0.578	0.778	0.549	0.740	0.531	0.863	0.584
Chroococcales	<i>Chroococcus</i>	0.695	0.471	0.688	0.457	0.732	0.487	0.695	0.457
	<i>Woronichinia</i>	0.860	0.642	0.828	0.539	0.869	0.687	0.844	0.579
Oscillatoriales	<i>Phormidium</i>	0.673	0.454	0.641	0.428	0.617	0.427	0.686	0.454
	<i>Lyngbya</i>	0.890	0.547	0.889	0.577	0.912	0.573	0.832	0.535
Total		0.792	0.532	0.773	0.516	0.789	0.540	0.790	0.526

Table 5. Object identification results for different cyanobacteria genera using the YOLOv8 architecture on a desktop computer with GPU acceleration (Pipeline 2).

Order	Genera	YOLOv8-s		YOLOv8-m		YOLOv8-l		YOLOv8-x	
		mAP50	mAP50:95	mAP50	mAP50:95	mAP50	mAP50:95	mAP50	mAP50:95
Nostocales	<i>Anabaena</i>	0.774	0.511	0.789	0.515	0.871	0.552	0.774	0.522
	<i>Dolicospermum</i>	0.848	0.580	0.789	0.567	0.709	0.507	0.868	0.609
Chroococcales	<i>Chroococcus</i>	0.696	0.466	0.679	0.457	0.717	0.475	0.683	0.456
	<i>Woronichinia</i>	0.876	0.645	0.828	0.520	0.868	0.685	0.844	0.588
Oscillatoriales	<i>Phormidium</i>	0.689	0.454	0.642	0.438	0.698	0.453	0.710	0.484
	<i>Lyngbya</i>	0.882	0.535	0.860	0.550	0.940	0.592	0.855	0.542
Total		0.794	0.532	0.765	0.508	0.800	0.544	0.789	0.534

4.3.2. Diatom Results

First, the evaluation metrics for the different versions of the YOLOv8 architecture were calculated using the diatom test split. Tables 6 and 7 present the *mAP50* and *mAP50:95* scores for each of the ten diatom genera considered in this study, along with the mean values across all classes, calculated using the two pipelines. These results offer a comprehensive view of how well the different YOLOv8 variants perform across various diatom genera, highlighting their strengths and weaknesses in terms of both detection accuracy and model robustness.

The results show that the YOLOv8-x architecture achieved the best average performance in terms of accuracy for both methodologies, demonstrating its superior capability in handling the complexity of diatom classification. Additionally, while other versions like YOLOv8-s also performed well, the differences in results were minimal, suggesting that the choice of architecture may depend on the specific trade-offs between accuracy and computational efficiency required for the task at hand. Figure 8 illustrates the inference on an image of the genus *Melosira* using the YOLOv8-s architecture and Pipeline 2 methodology, showcasing how the model processes and classifies diatom specimens in a real-world scenario.

For Pipeline 1, the YOLOv8-x architecture achieved the highest values of *mAP50* (0.841) and *mAP50:95* (0.715), resulting in an average value of 0.778. For Pipeline 2, the YOLOv8-x architecture also obtained the highest value of *mAP50* (0.839) and *mAP50:95* (0.713), yielding an average value of 0.776. The differences between the various YOLOv8 architectures are minimal, with the YOLOv8-s architecture attaining the lowest values and the YOLOv8-x architecture the highest in Pipeline 1. In Pipeline 2, the YOLOv8-m architecture reached the lowest values, while the YOLOv8-x architecture again achieved the highest values. Interestingly, in this pipeline, the smallest architecture, YOLOv8-s, achieved the second-highest values of *mAP50* and *mAP50:95*, demonstrating that smaller architectures can still perform competitively under certain conditions. Figure 9 shows the inference on an image of the genus *Cymbella* using the YOLOv8-s architecture and the Pipeline 1 methodology.

In contrast to the case of cyanobacteria, it is evident that increasing the network depth results in higher *mAP* values for diatoms. This improvement can be attributed to the diatom dataset's larger size, as well as the greater diversity in the taxonomy of diatoms. As shown in Figure 6, the diatom dataset contains a broader variety of species, contributing to better performance and higher *mAP* values compared to the cyanobacteria dataset, where the smaller dataset size and more similar genera may limit model performance.

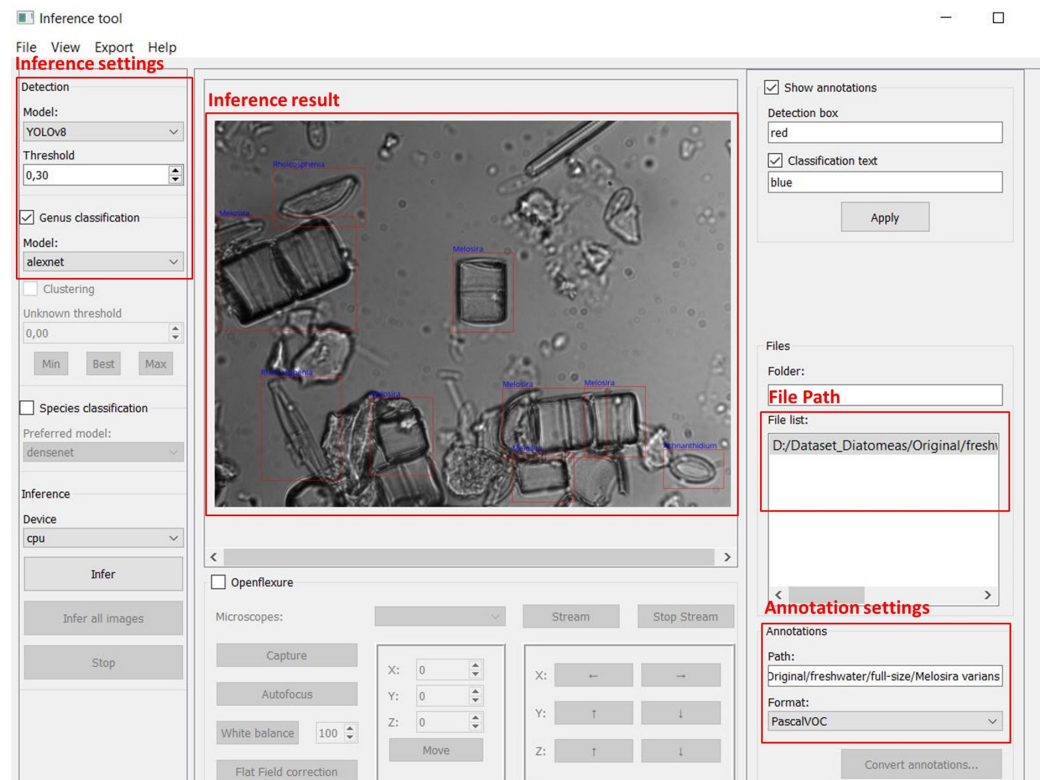


Figure 8. Inference using YOLOv8-s and pipeline 2 on a image of the genus *Melosira*.

Table 6. Object identification results for different diatom genera using the YOLOv8 architecture on the HAILO-8L accelerator (Pipeline 1).

Genera	YOLOv8-s		YOLOv8-m		YOLOv8-l		YOLOv8-x	
	mAP50	mAP50:95	mAP50	mAP50:95	mAP50	mAP50:95	mAP50	mAP50:95
<i>Aulacoseira</i>	0.668	0.558	0.696	0.580	0.762	0.640	0.766	0.644
<i>Cocconeis</i>	0.837	0.734	0.870	0.762	0.851	0.758	0.852	0.760
<i>Cymbella</i>	0.910	0.759	0.921	0.778	0.922	0.780	0.911	0.772
<i>Diatoma</i>	0.870	0.730	0.911	0.759	0.901	0.757	0.914	0.778
<i>Epithemia</i>	0.860	0.716	0.887	0.739	0.882	0.739	0.869	0.735
<i>Gomphonema</i>	0.805	0.691	0.819	0.698	0.857	0.742	0.855	0.743
<i>Melosira</i>	0.753	0.611	0.764	0.625	0.759	0.619	0.763	0.632
<i>Planothidium</i>	0.742	0.640	0.761	0.658	0.812	0.701	0.838	0.731
<i>Staurosira</i>	0.767	0.656	0.786	0.677	0.797	0.688	0.816	0.708
<i>Tabellaria</i>	0.818	0.646	0.808	0.630	0.816	0.633	0.824	0.649
Total	0.803	0.674	0.822	0.691	0.836	0.706	0.841	0.715

Table 7. Object identification results for different diatom genera using the YOLOv8 architecture on a desktop computer with GPU acceleration (Pipeline 2).

Genera	YOLOv8-s		YOLOv8-m		YOLOv8-l		YOLOv8-x	
	mAP50	mAP50:95	mAP50	mAP50:95	mAP50	mAP50:95	mAP50	mAP50:95
<i>Aulacoseira</i>	0.725	0.599	0.711	0.591	0.75	0.625	0.761	0.637
<i>Cocconeis</i>	0.847	0.748	0.864	0.765	0.855	0.761	0.849	0.756
<i>Cymbella</i>	0.912	0.759	0.912	0.768	0.918	0.777	0.915	0.771
<i>Diatoma</i>	0.870	0.737	0.904	0.763	0.874	0.736	0.905	0.766
<i>Epithemia</i>	0.847	0.712	0.869	0.734	0.875	0.737	0.867	0.735
<i>Gomphonema</i>	0.827	0.713	0.804	0.695	0.841	0.736	0.853	0.745
<i>Melosira</i>	0.756	0.614	0.772	0.638	0.770	0.625	0.770	0.638
<i>Planothidium</i>	0.797	0.684	0.735	0.641	0.745	0.643	0.832	0.728
<i>Staurosira</i>	0.823	0.710	0.762	0.663	0.777	0.672	0.812	0.712
<i>Tabellaria</i>	0.831	0.635	0.822	0.632	0.785	0.628	0.823	0.642
Total	0.824	0.691	0.816	0.689	0.819	0.694	0.839	0.713

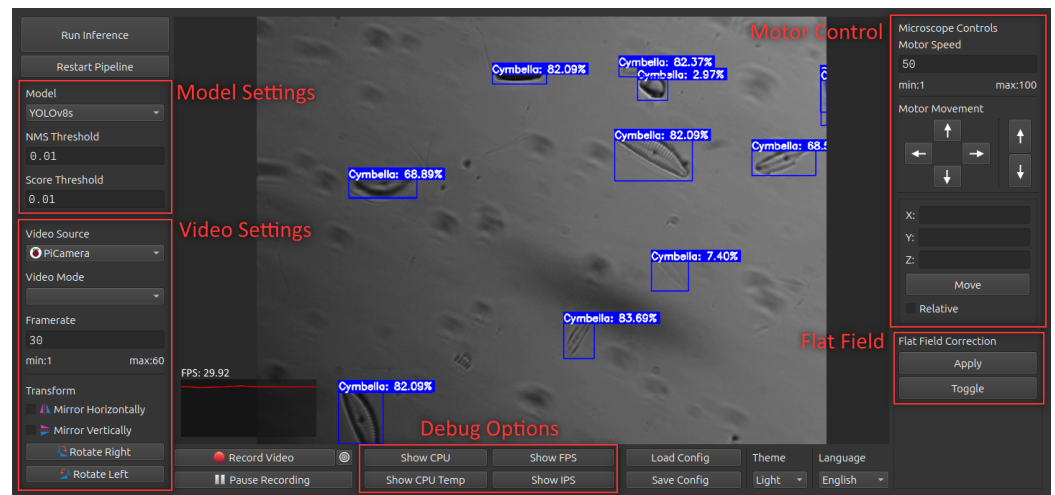


Figure 9. Inference using YOLOv8-s and Pipeline 1 on a image of the genus *Cymbella*.

Although there are minimal differences in performance compared to the non-compiled models due to the optimization process, the accuracy across both pipelines can be considered equivalent. This indicates that the results obtained from the HAILO-8L accelerator on the Raspberry Pi 5 can be competitive, even in scenarios traditionally dominated by higher-performance desktop setups. Such a comparison highlights the potential of edge devices to deliver effective and efficient solutions in resource-constrained environments, providing an attractive option for real-time applications in microscopy and similar fields.

4.4. Video Streaming

Both pipelines allow the user to interact with the microscope stage by controlling the platform motors in real time. Ideally, any change in the environment caused by user actions should be communicated back as quickly as possible to ensure a seamless and responsive interaction.

For most video streaming applications, frame rates above 30 FPS provide a smooth viewing experience, corresponding to frame times below 33.33 ms. Additionally, the latency between user input and received feedback should remain below 100 ms to maintain real-time responsiveness.

In this context, we define a real-time video application as one capable of displaying a live feed of the environment at a consistent 30 FPS, with a maximum end-to-end latency of 100 ms.

Pipeline 1: The detection application operates with a video resolution of 1640×1232 , which is downsampled to 832×624 at 42 FPS. Frame rate measurements indicate an average output of 41.94 FPS with a standard deviation of 0.55. Latency measurements show an average of 53.3 ms, with a margin of error of 4.2 ms.

These results indicate that Pipeline 1 is fully capable of delivering real-time user interaction in terms of video streaming, frame rate, and latency. This remains true even when inference is enabled, as inference and video streaming operate as separate, decoupled processes. Consequently, models with inference times longer than the frame time of the video recording device do not affect the display frame rate.

Pipeline 2: Pipeline 2 is not designed for real-time interaction due to its inherent characteristics. It requires a static image for classification, meaning that an image must be loaded or captured from the microscope camera before applying the inference models. However, it is capable of maintaining a constant average frame rate of 16 FPS, with a standard deviation of 1 FPS. Latency depends on network saturation between the

microscope and the computer; in this case, it is approximately 1.50 s, with a margin of error of 0.20 s.

These values indicate that Pipeline 2 does not achieve real-time interaction but remains a valuable tool for experts, helping to reduce labeling time and minimize human error in classification.

4.5. Inference Time

Another important metric to consider when using hardware accelerators like the HAILO-8L is inference time, as it directly impacts the practical usability of the system. Table 8 presents the number of frames per second (FPS) achieved by different versions of the YOLOv8 architecture on the HAILO-8L accelerator and the Raspberry Pi 5's CPU.

For Case 1, Pipeline 1's Detection Application was tested using the default video configuration of 832×624 resolution at 42 FPS. In contrast, Cases 2 and 3 were evaluated with randomly generated data, utilizing the Hailo SDK Python library and the PyTorch library, respectively.

The results indicate that the YOLOv8-s architecture achieved the best performance in terms of speed, with a frame rate of 50.72 FPS, making it highly suitable for real-time applications. In contrast, the YOLOv8-x architecture demonstrated the lowest performance, with a frame rate of 8.53 FPS. However, this performance is still within acceptable limits for many real-time applications, especially given the complexity of the YOLOv8-x architecture and the large number of parameters it involves. These findings underscore the viability of using hardware accelerators like the HAILO-8L for various microscopy-related applications, offering a balance between computational power and real-time processing capabilities.

Table 8. FPS comparison obtained for the different versions of the YOLOv8 architecture on the HAILO-8L accelerator and on the Raspberry Pi 5's CPU using PyTorch.

Case	YOLOv8-s	YOLOv8-m	YOLOv8-l	YOLOv8-x
1 (APP)	41.92	21.90	11.91	7.95
2 (HAILO)	50.72	25.58	13.37	8.53
3 (CPU)	0.99	0.41	0.20	0.14

The results for Cases 2 and 3, shown in Table 8, were obtained by running the models on previously generated input data matching the required dimensions of each model. The experiments were conducted using Hailo's SDK Python library and PyTorch.

In real-world applications, additional computational overhead may be introduced, potentially stalling or reducing inference performance. Consequently, the performance metrics reported in Cases 2 and 3 may surpass those achievable in practical deployments, as demonstrated in Case 1.

Our primary objective for Pipeline 1 was to detect as many cyanobacteria and diatoms as possible while the motors moved the stage across the sample. If the inference time was too slow, certain areas of the sample might not be captured or processed by the model, leading to undetected cyanobacteria regardless of model accuracy.

By achieving low inference times and leveraging digitally controlled motors, this approach can be further developed into a fully automated system. The microscope could be scripted to systematically scan a predefined sample region, generating a comprehensive report based on the extracted data.

After manually testing each model, we conclude that Pipeline 1 provides sufficient inference speed to effectively scan different regions of the sample while maintaining strong detection performance. The Raspberry Pi camera's exposure time was set to 1.75 ms to

minimize motion blur, ensuring clear images during movement. As a result, the models maintain high performance even while the stage is in motion.

4.6. Energy Efficiency Analysis

To evaluate Pipeline 1's power efficiency, several experiments were conducted under different workloads. A total of seven distinct test cases were analyzed. For each case, the following measurements were recorded:

- CPU usage by Pipeline 1's application (in %).
- Hailo Device Utilization (in %), measured using the Hailo Runtime API.
- Total Power Consumption (in watts), measured using a USB power meter connected to the Raspberry Pi 5's power supply.
- CPU Temperature (in Celsius).
- FPS (Inferenced Frames Per Second): This metric measures the number of frames processed by the inference model per second. The total number of frames displayed by the application depends on the video device and the selected video configuration.

For each case, 20 samples were collected over a 10-minute period and the average values were calculated. Each case builds upon the previous one, either enabling additional hardware or running more demanding software. The test cases are as follows:

1. **Idle RPi5:** This case represents the power consumption of the Raspberry Pi 5 when idle, with no application running and no Sangaboard (motors or LEDs) connected. CPU usage is effectively zero, and the Hailo device is not connected. Only the operating system is running.
2. **Idle RPi5 and Idle Hailo:** Similar to the previous case, but with the Hailo device connected (but not in use).
3. **Running Hailo:** In this case, no application is running and the motors and LED are not connected, but the Hailo device is connected and used for inference. Inference is performed using the Hailo Runtime tools, not the detection application developed for Pipeline 1.
4. **Idle Hailo and Sangaboard:** This case is identical to case 2 but with motors and LEDs connected. This case is used to measure the power consumption of the motors and LEDs.
5. **Running Application:** This case represents the power consumption of the Raspberry Pi 5 when the application for Pipeline 1 is running, but no inference is being performed.

Cases 6 to 9 represent the main use case, where the Sangaboard is connected, the application is running and inference is performed using the four available models. For Cases 5 to 9, the Detection Application was executed using a video configuration of 1640×1232 , downsampled to 832×624 at 42 FPS. To assess power consumption at maximum performance, the application was configured without frame rate limitations. However, power consumption can be reduced by capping the frame rate according to user preferences.

Unfortunately, direct measurement of the Hailo device's power consumption was not possible due to hardware limitations. Therefore, its power consumption was estimated based on the utilization metric.

First, Cases 1 and 2 were tested and compared to estimate the Hailo device's power consumption when idle (static consumption), which was found to be 0.20 watts. Then, Cases 2 and 3 were compared to estimate the Hailo device's power consumption during inference at 100% utilization (dynamic consumption). Case 3 showed a 1.32-watt increase in power consumption compared to Case 2.

The following formula was used to calculate the Power Consumption (PWC) of the Hailo device:

$$\text{Hailo PWC} = \text{Static Consumption} + (\text{Hailo Device Utilization} * \text{Dynamic Consumption}).$$

Substituting the values:

$$\text{Hailo PWC} = 0.20 + (\text{Hailo Device Utilization} * 1.32).$$

This formula yields an estimated power consumption range of 0.20 to 1.52 watts, depending on the device's utilization. This estimate aligns with the "typical power consumption of 1.5 watts" specified in the device datasheet.

The power consumption of the application was estimated using the following formula:

$$\text{Application PWC} = \text{Total PWC} - \text{RPi5 PWC} - \text{Hailo PWC} - \text{Motor PWC} - \text{LEDs PWC}.$$

Finally, Cases 4 through 9 were tested, and the results are presented in Table 9.

Table 9. Power consumption and usage metrics for different test cases using the Raspberry Pi 5 and Hailo accelerator (Pipeline 1).

Case	Usage (%)		Power Consumption (W)			Temp (°C)	FPS
	CPU	Hailo	App PWC	Hailo PWC	Total PWC		
1. Idle RPi5	0.0	0.0	0.0	0.0	4.98	50	0
2. Idle RPi5 and Idle Hailo	0.0	0.0	0.0	0.20	5.18	50	0
3. Running Hailo	0.0	100	0.0	1.52	6.50	50	50.42
4. Idle Hailo and Sangaboard	0.0	0.0	0.0	0.0	6.07	50	0
5. Running Application	30.0	0.0	2.80	0.0	8.93	56	0
6. YOLOv8-s	44.4	69.5	3.33	1.12	10.52	58	41.92
7. YOLOv8-m	37.5	80.1	3.23	1.25	10.55	58	21.90
8. YOLOv8-l	36.2	90.0	3.11	1.38	10.56	57	11.91
9. YOLOv8-x	34.3	94.0	2.90	1.44	10.41	56	7.95

The results of the energy performance analysis in Pipeline 1 indicate that Hailo device usage decreases when simpler models are executed, while CPU utilization increases (see Figure 10). This behaviour occurs primarily due to two reasons:

1. Each frame is individually resized to match the model's input dimensions before inference is performed. This step is executed by the CPU, causing the Hailo device to remain idle until the data are ready for inference. Consequently, this idle time prevents the Hailo device from operating at full capacity. The maximum obtainable utilization can be calculated using the following equation:

$$\text{Maximum Utilization} = \frac{\text{Inference Time}}{\text{Inference Time} + \text{Resize Time}}.$$

2. For each completed inference, the pipeline triggers a signal to redraw the bounding boxes based on the latest inference data. This operation is handled by the CPU. Faster models, which generate more output data, increase the frequency of redraw events, thereby leading to higher CPU usage.

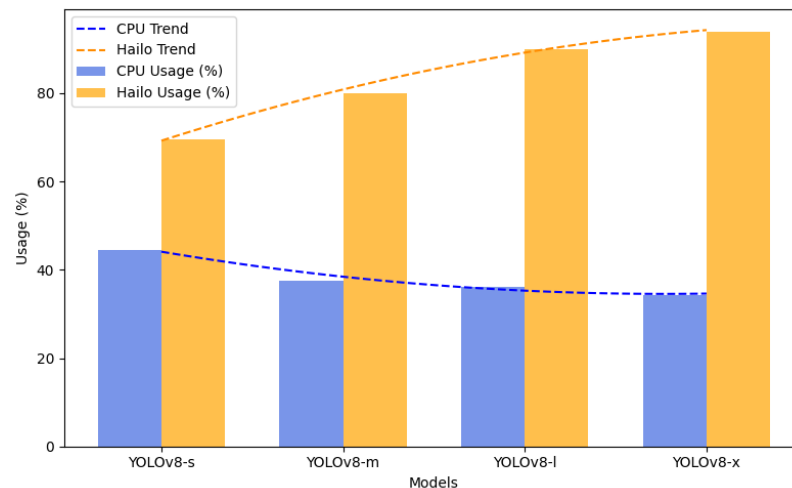


Figure 10. Comparison of CPU and Hailo-8L usage metrics for Pipeline 1.

Extensive testing of Pipeline 1 has demonstrated high power efficiency, with successful inference across all tested models. Power consumption remains within a narrow range of 10.4 to 10.6 watts, while maintaining moderate operating temperatures. These results indicate high energy efficiency, making Pipeline 1 well-suited for environments with strict power constraints, even when running at full capacity.

Figure 11 illustrates the distribution of power consumption across different pipeline stages tested under various workloads. Note that the values for Inference Consumption and the Detection Application are estimates, intended solely to provide an approximate representation of the computational workload at each pipeline stage.

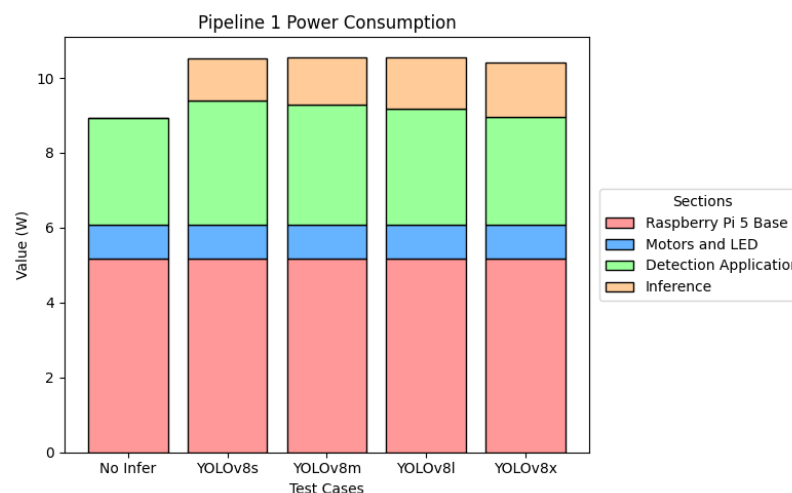


Figure 11. Bar-Chart for power consumption of Pipeline 1.

To analyze the power efficiency of Pipeline 2, the power consumption of its various components was considered:

- CPU usage by Pipeline 2's application (in %).
- CPU and GPU used (in %) by Pipeline 2 on the desktop computer.
- Power consumption (in watts), of the Raspberry Pi 4, measured using a USB power meter.
- Watt consumption of the Sangaboard v02 used for motor movement and lighting.
- Power consumption of the CPU and GPU resources used in the desktop computer.

Several samples were collected over a 10-minute period, and the average values were calculated. The test cases are as follows:

1. **Idle RPi4:** This case represents the power consumption of the Raspberry Pi 4 when idle. The OpenFlexure Connect software runs in the background when the Raspberry Pi is powered on, resulting in CPU usage greater than zero.
2. **Idle RPi4 with desktop application running:** In this case, the desktop application is running and connected to the OpenFlexure via cable.
3. **Running application with Sangaboard connected:** This case is similar to the previous one, but the microscope Sangaboard is also connected.

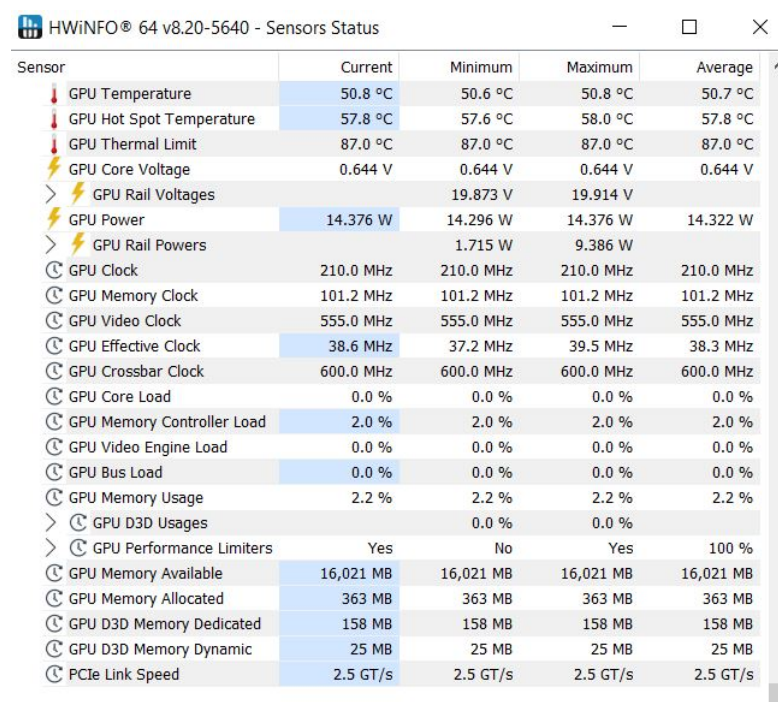
Cases 4 to 7 represent the main use case, where the Sangaboard is connected, the application is running, and inference is performed using different models. In Pipeline 2, the FPS remains stable at an average of 14, with latency as mentioned in the previous section.

In Case 1, Raspberry Pi power consumption was measured using a USB power meter connected to the Raspberry Pi 4 power supply, showing an average idle power consumption of 2.789 W.

In Case 3, the Sangaboard is powered in parallel with the Raspberry Pi 4, allowing its power consumption to be determined using the voltage measured by the power meter. The average Raspberry Pi 4 idle power consumption remains 2.789 W.

Since each stepper motor draws approximately 55 mA, and the power supply voltage is 5.4 V, the Sangaboard's power consumption ranges from 0.297 W (when only one motor is active) to 0.837 W (when all motors are running).

In Cases 2 and 3, CPU and GPU usage, as well as power consumption on the desktop computer, were measured using tools such as NVIDIA System Management Interface, Windows Task Manager, and HWiNFO diagnostic software (see Figure 12).



Sensor	Current	Minimum	Maximum	Average
GPU Temperature	50.8 °C	50.6 °C	50.8 °C	50.7 °C
GPU Hot Spot Temperature	57.8 °C	57.6 °C	58.0 °C	57.8 °C
GPU Thermal Limit	87.0 °C	87.0 °C	87.0 °C	87.0 °C
GPU Core Voltage	0.644 V	0.644 V	0.644 V	0.644 V
GPU Rail Voltages		19.873 V	19.914 V	
GPU Power	14.376 W	14.296 W	14.376 W	14.322 W
GPU Rail Powers		1.715 W	9.386 W	
GPU Clock	210.0 MHz	210.0 MHz	210.0 MHz	210.0 MHz
GPU Memory Clock	101.2 MHz	101.2 MHz	101.2 MHz	101.2 MHz
GPU Video Clock	555.0 MHz	555.0 MHz	555.0 MHz	555.0 MHz
GPU Effective Clock	38.6 MHz	37.2 MHz	39.5 MHz	38.3 MHz
GPU Crossbar Clock	600.0 MHz	600.0 MHz	600.0 MHz	600.0 MHz
GPU Core Load	0.0 %	0.0 %	0.0 %	0.0 %
GPU Memory Controller Load	2.0 %	2.0 %	2.0 %	2.0 %
GPU Video Engine Load	0.0 %	0.0 %	0.0 %	0.0 %
GPU Bus Load	0.0 %	0.0 %	0.0 %	0.0 %
GPU Memory Usage	2.2 %	2.2 %	2.2 %	2.2 %
GPU D3D Usages		0.0 %	0.0 %	
GPU Performance Limiters	Yes	No	Yes	100 %
GPU Memory Available	16,021 MB	16,021 MB	16,021 MB	16,021 MB
GPU Memory Allocated	363 MB	363 MB	363 MB	363 MB
GPU D3D Memory Dedicated	158 MB	158 MB	158 MB	158 MB
GPU D3D Memory Dynamic	25 MB	25 MB	25 MB	25 MB
PCIe Link Speed	2.5 GT/s	2.5 GT/s	2.5 GT/s	2.5 GT/s

Figure 12. Values of the GPU usage and consumption provided by the HWiNFO diagnostic software.

To obtain the GPU values, inference was performed using different models, and GPU usage and power consumption were measured. As shown in Figure 12, the GPU has a baseline usage and power consumption, which has been subtracted from the values obtained during inference:

$$GPU\ usage = GPU\ current\ usage - 2$$

$$GPU\ Power\ Consumption = GPU\ current\ Power\ Consumption - 14.7.$$

The desktop computer's CPU is also used to run the application, capture images, load the model, and draw the bounding boxes. Additionally, the application allows users to select the device for running inference. In this case, the model is loaded using the CPU and then transferred to the GPU for inference.

The same steps were followed to measure CPU power consumption:

$$\text{CPU Power Consumption} = \text{CPU current Power Consumption} - 9.$$

However, to obtain CPU usage, the Task Manager was used, as the CPU usage values provided by HWiNFO (version 8.20-5640) diagnostic software were not accurate. The estimated usage and power consumption values for Pipeline 2 are presented in Table 10.

Figure 13 illustrates the distribution of power consumption across the different stages of Pipeline 2. The highest power consumption occurs in the desktop computer, while the Raspberry Pi accounts for only about 10% of the total power consumption.

Overall, Pipeline 2 consumes five times more power than Pipeline 1, despite producing similar inference results across all models.

Table 10. Power consumption and usage metrics for different test cases using the Raspberry Pi4 and the desktop computer (Pipeline 2).

Case	Usage (%)			Power Consumption (W)				Temp (°C)
	RPi4 CPU	DC CPU	DC GPU	Rpi4	DC CPU	DC GPU	Total PWC	
1. Idle RPi4	3	0.0	0.0	2.78	0.0	0.0	2.78	39
3. Running App	5.3	0.0020	0.0	2.81	0.0	0.0	2.81	44
5. Running App & Sanga	10	0.0042	0.0	3.17	0.0	0.0	3.17	46
6. YOLOv8-s	10	0.9	1.5	3.17	23.0	19.8	45.97	46
7. YOLOv8-m	10	1	1.9	3.17	23.3	20.2	46.7	46
8. YOLOv8-l	10	1.3	2.5	3.17	23.9	21.0	48.1	46
9. YOLOv8-x	10	1.5	3.1	3.17	25.4	21.6	50.2	46

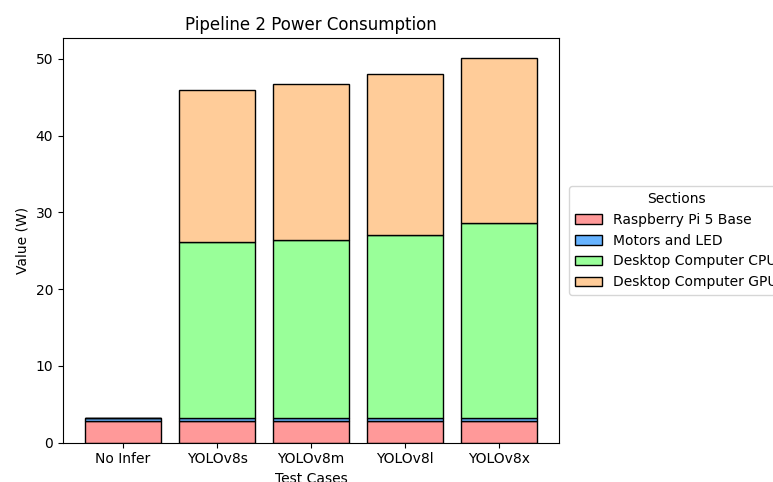


Figure 13. Bar chart for power consumption of Pipeline 2.

5. Conclusions

In this work, two software pipelines for real-time image processing in low-cost microscopy solutions using deep learning architectures are proposed. The first pipeline leverages edge computing for real-time processing by utilizing the OpenFlexure Microscope with the HAILO-8L accelerator, while the second pipeline is based on the OpenFlexure Delta Stage paired with a desktop computer featuring GPU acceleration. Both pipelines were evaluated using a dataset of microscopy images of cyanobacteria and diatom samples, employing the YOLOv8 architecture for object detection and classification. Experiments

conducted on the diatom dataset demonstrate that, in terms of classification, the HAILO-8L accelerator outperforms the GPU acceleration.

Despite the relatively small size of the datasets and the complexity of the task due to the high similarity between genera, the results demonstrate that the YOLOv8 architecture delivers strong performance in both accuracy and speed. The evaluation of both pipelines across these two use cases—cyanobacteria and diatoms—shows that, while the performance is comparable in both cases, the HAILO-8L accelerator (Pipeline 1, Tables 4 and 6) delivers similar results to a desktop computer with GPU acceleration (Pipeline 2, Tables 5 and 7). This is particularly promising for edge computing applications, where real-time microscopic analysis is required in situ using portable and low-cost microscopes.

These findings highlight the potential of edge computing solutions to support a wide range of practical microscopy applications in resource-constrained environments. While the HAILO-8L accelerator demonstrates comparable performance to desktop GPU setups in many scenarios, its limitations, such as support for only a restricted set of deep learning architectures, must be considered. In contrast, desktop computers with GPU acceleration offer greater flexibility, enabling a broader range of models, the deployment of custom solutions and the use of more complex architectures. Nonetheless, the portability and cost-effectiveness of edge computing make it a viable option for resource-constrained settings, particularly when applications do not require extensive model complexity.

Since its introduction, the Hailo 8L accelerator has been used in a wide range of applications, including automotive, security, and industrial automation. In this article, we aim to demonstrate the potential of the Hailo 8L accelerator in low-cost edge devices. This approach enables countries and research teams with limited resources to perform tasks that previously required expensive, energy-intensive equipment. Very little research has been conducted in this area, and we expect that the promising results achieved in this study will contribute to the further development of low-cost edge devices, making artificial intelligence more accessible.

Future work should explore the trade-offs between edge computing and desktop GPU setups to optimize their application in cyanobacteria and diatom microscopy, as well as similar tasks. This includes further experiments to assess pipeline performance across varied scenarios and datasets, expanding to other microscopy applications for broader insights and incorporating data augmentation to enhance dataset diversity. Investigating alternative deep learning architectures could also improve model performance and adaptability for resource-constrained environments.

Exploring alternative deep learning architectures could further improve model performance and adaptability to resource-constrained environments. For instance, more modern architectures, such as YOLOX, may enhance detection accuracy in less distinct and smaller datasets, like those used in this experiment. This architecture is also less prone to overfitting in such scenarios. Other architectures, particularly those based on few-shot learning mechanisms, address the challenge of small datasets by reducing network depth, enhancing feature extraction, and incorporating classification methods based on machine learning algorithms.

Another potential extension of this work could involve integrating environmental analysis techniques that use fluorescent imaging. This capability could provide deeper insights into the sample's characteristics, enhancing the quality and reliability of the data collected across both processing pipelines.

The scope of this work could also be expanded to other domains, such as pathological microscopy, where the same framework could be used to automate the detection and counting of different cell types in tissue samples.

As mentioned earlier, a potential improvement for Pipeline 1 is automation. The results demonstrated that Pipeline 1 maintains consistent detection performance even when the stage is in motion, while achieving low inference times. Automating the device would enable systematic scanning and data collection across a sample region.

By applying advanced data processing techniques, it would be possible to generate comprehensive reports, including the number of detections per class and the spatial distribution of detected objects across the scanned regions. This approach could be extended to various applications involving object detection with digital microscopes, such as parasite identification, cell analysis, and cancer detection, providing valuable insights for scientific and medical research.

Author Contributions: Conceptualization, G.B. and G.C.; methodology, G.B. and J.R.-S.; software, L.S.-V., A.D.-M. and M.B.; validation, L.S.-V. and A.D.-M.; formal analysis, all authors; investigation, all authors; resources, G.B. and G.C.; data curation, all authors; writing—original draft preparation, G.B., J.R.-S., L.S.-V. and A.D.-M.; writing—review and editing, all authors; supervision, G.B., J.R.-S. and J.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research was partially funded by the DIAMOND project (Ref. TED2021-132147B-100), supported by the Spanish Ministry of Science, Innovation, and Universities and by the European Union NextGenerationEU/PRTR, and by project Horizon Europe dAIEdge Grant n. 101120726 by the European Commission.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding author.

Acknowledgments: We thank R. Bowman for his support and insights in developing the OpenFlexure microscope project.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

AI	Artificial Intelligence
AP	Average Precision
API	Application Programming Interface
BOM	Bill of Materials
COCO	Common Objects in Context (dataset and annotation format)
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture (NVIDIA's parallel computing platform)
DIC	Differential Interference Contrast
DNS	Domain Name System
FPS	Frames Per Second
GPU	Graphics Processing Unit
HAT	Hardware Attached on Top
IoU	Intersection over Union
ISP	Image Signal Processor
JSON	JavaScript Object Notation
MAIScope	Malaria Intelligent Scope
MIPI	Mobile Industry Processor Interface
mAP	Mean Average Precision
mAP50	mAP calculated with an IoU threshold of 0.5
mAP50:95	mAP calculated at IoU thresholds ranging from 0.5 to 0.95 (in increments of 0.05)
PWC	Power Consumption

RPi	Raspberry Pi
RGB	Red, Green, Blue (color model)
ROI	Region of Interest
SGD	Stochastic Gradient Descent (gradient descent optimization method)
STL	Standard Tessellation Language (file format for 3D printing)
TOPS	Tera Operations Per Second
TPU	Tensor Processing Unit

References

1. Garcia-Pardo, M.E.; Simpson, J.C.; O'Sullivan, N.C. A novel automated image analysis pipeline for quantifying morphological changes to the endoplasmic reticulum in cultured human cells. *BMC Bioinform.* **2021**, *22*, 427. [\[CrossRef\]](#)
2. Kallepalli, A.; Viani, L.; Stellinga, D.; Rotunno, E.; Bowman, R.; Gibson, G.M.; Sun, M.-J.; Rosi, P.; Frabboni, S.; Balboni, R.; et al. Challenging point scanning across electron microscopy and optical imaging using computational imaging. *Intell. Comput.* **2022**, *2022*, 0001. [\[CrossRef\]](#)
3. Ruiz-Santaquiteria, J.; Pedraza, A.; Vallez, N.; Velasco, A. Parasitic egg detection with a deep learning ensemble. In Proceedings of the 2022 IEEE International Conference on Image Processing (ICIP), Bordeaux, France, 16–19 October 2022; pp. 4283–4286.
4. Knapper, J.; Whiteford, F.; Rosen, D.; Wadsworth, W.; Stirling, J.; Mkindi, C.; Mduda, J.; Sanga, V.L.; Nyakyi, P.T.; Mboa Nkoudou, T.H.; et al. Developing the Openflexure microscope towards medical use: Technical and social challenges of developing globally accessible hardware for Healthcare. *Philos. Trans. A* **2024**, *382*, 20230257. [\[CrossRef\]](#) [\[PubMed\]](#)
5. Bueno, G.; Sanchez, L.; Perona, E.; Muñoz-Martín, M.A.; Hiruelas, A.; Salido, J.; Cristobal, G. Microscopic image quality in few-shot GAN-generated cyanobacteria images and its impact on classification networks. In Proceedings of the Optics, Photonics, and Digital Technologies for Imaging Applications VIII, Strasbourg, France, 9–11 April 2024; Volume 12998, pp. 40–56. [\[CrossRef\]](#)
6. Mysior, M.M.; Simpson, J.C. An automated high-content screening and assay platform for the analysis of spheroids at subcellular resolution. *PLoS ONE* **2024**, *19*, e0311963. [\[CrossRef\]](#)
7. Salido, J.; Bueno, G.; Ruiz-Santaquiteria, J.; Cristobal, G. A review on low-cost microscopes for Open Science. *Microsc. Res. Tech.* **2022**, *85*, 3270–3283. [\[CrossRef\]](#) [\[PubMed\]](#)
8. Moreno-Roman, P.; Bobick, K. Foldscope: Increasing science accessibility worldwide. *Microsc. Today* **2022**, *30*, 42–45. [\[CrossRef\]](#)
9. Cybulski, J.S.; Clements, J.; Prakash, M. Foldscope: Origami-based paper microscope. *PLoS ONE* **2014**, *9*, e98781. [\[CrossRef\]](#)
10. Sangameswaran, R. MAIScope: A low-cost portable microscope with built-in vision AI to automate microscopic diagnosis of diseases in remote rural settings. *arXiv* **2022**, arXiv:2208.06114.
11. Grignaffini, F.; Simeoni, P.; Alisi, A.; Frezza, F. Computer-Aided Diagnosis Systems for Automatic Malaria Parasite Detection and Classification: A Review. *Electronics* **2024**, *13*, 3174. [\[CrossRef\]](#)
12. Skandarajah, A.; Reber, C.D.; Switz, N.A.; Fletcher, D.A. Quantitative imaging with a mobile phone microscope. *PLoS ONE* **2014**, *9*, e96906. [\[CrossRef\]](#)
13. Diederich, B.; Lachmann, R.; Carlstedt, S.; Marsikova, B.; Wang, H.; Uwurukundo, X.; Mosig, A.S.; Heintzmann, R. A versatile and customizable low-cost 3D-printed open standard for microscopic imaging. *Nat. Commun.* **2020**, *11*, 5979. [\[CrossRef\]](#) [\[PubMed\]](#)
14. Salido, J.; Toledano, P.; Vallez, N.; Deniz, O.; Ruiz-Santaquiteria, J.; Cristóbal, G.; Bueno, G. MicroHikari3D: An automated DIY digital microscopy platform with deep learning capabilities. *Biomed. Opt. Express* **2021**, *12*, 7223–7243. [\[CrossRef\]](#) [\[PubMed\]](#)
15. Collins, J.T.; Knapper, J.; Stirling, J.; Mduda, J.; Mkindi, C.; Mayagaya, V.; Mwakajinga, G.A.; Nyakyi, P.T.; Sanga, V.L.; Carbery, D.; et al. Robotic microscopy for everyone: The OpenFlexure microscope. *Biomed. Opt. Express* **2020**, *11*, 2447–2460. [\[CrossRef\]](#) [\[PubMed\]](#)
16. Dabrowska, A.; Lewis, G.R.; Atlabachew, M.; Salter, S.J.; Henderson, C.; Ji, C.; Ehlers, A.; Stirling, J.; Mower, S.; Allen, L.; et al. Expanding access to water quality monitoring with the open-source WaterScope testing platform. *npj Clean Water* **2024**, *7*, 68. [\[CrossRef\]](#)
17. Bowman, R.; Müllenbroich, C.; Diederich, B.; McConnell, G.; Faez, S.; Arancio, J. Open, reproducible hardware for microscopy. *Philos. Trans. A Math. Phys. Eng. Sci.* **2024**, *382*, 20230112. [\[CrossRef\]](#) [\[PubMed\]](#)
18. McDermott, S.; Kotar, J.; Collins, J.; Mancini, L.; Bowman, R.; Cicutta, P. Using old laboratory equipment with modern Web-of-Things standards: A smart laboratory with LabThings Retro. *R. Soc. Open Sci.* **2024**, *11*, 240634. [\[CrossRef\]](#) [\[PubMed\]](#)
19. Salido, J.; Sánchez, C.; Ruiz-Santaquiteria, J.; Cristóbal, G.; Blanco, S.; Bueno, G. A low-cost automated digital microscopy platform for automatic identification of diatoms. *Appl. Sci.* **2020**, *10*, 6033. [\[CrossRef\]](#)
20. Ruiz-Santaquiteria, J.; Bueno, G.; Deniz, O.; Vallez, N.; Cristobal, G. Semantic versus instance segmentation in microscopic algae detection. *Eng. Appl. Artif. Intell.* **2020**, *87*, 103271. [\[CrossRef\]](#)
21. Google. Coral TPU USB Accelerator. Available online: <https://coral.ai/products/accelerator/> (accessed on 6 January 2025).
22. Hailo. Hailo-8L Accelerator. Available online: <https://hailo.ai/products/ai-accelerators/hailo-8l-ai-accelerator-for-ai-light-applications/#hailo8l-overview> (accessed on 6 January 2025).

23. Bowman, R.W. Improving instrument reproducibility with open source hardware. *Nat. Rev. Methods Prim.* **2023**, *3*, 27. [[CrossRef](#)]
24. Howard, A.G.; Zhu, M.; Chen, B.; Kalenichenko, D.; Wang, W.; Weyand, T.; Andreetto, M.; Adam, H. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *arXiv* **2017**, arXiv:1704.04861. [[CrossRef](#)]
25. He, K.; Zhang, X.; Ren, S.; Sun, J. Deep residual learning for image recognition. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 770–778. [[CrossRef](#)]
26. Redmon, J.; Farhadi, A. YOLOv3: An Incremental Improvement. *arXiv* **2018**, arXiv:1804.02767. [[CrossRef](#)]
27. Grekov, A.N.; Shishkin, Y.E.; Peliushenko, S.S.; Mavrin, A.S. Application of the YOLOv5 Model for the Detection of Microobjects in the Marine Environment. *arXiv* **2022**, arXiv:2211.15218. [[CrossRef](#)]
28. Li, X.; Gu, S.; Wei, D. Plankton detection based on improved YOLOv5. In Proceedings of the International Conference on Optics and Machine Vision (ICOMV 2023), Changsha, China, 6–8 January 2023; Volume 4, p. 25. [[CrossRef](#)]
29. Jocher, G.; Chaurasia, A.; Qiu, J. YOLOv8 by Ultralytics. 2024. Available online: <https://github.com/ultralytics/ultralytics> (accessed on 6 January 2025).
30. Varghese, R.; Sambath, M. YOLOv8: A Novel Object Detection Algorithm with Enhanced Performance and Robustness. In Proceedings of the 2024 International Conference on Advances in Data Engineering and Intelligent Computing Systems (ADICS), Chennai, India, 18–19 April 2024; pp. 1–6. [[CrossRef](#)]
31. Karras, T.; Aittala, M.; Hellsten, J.; Laine, S.; Lehtinen, J.; Aila, T. Training Generative Adversarial Networks with Limited Data. *arXiv* **2020**, arXiv:2006.06676. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.